

Docker

For Build Image

step 1 :- For build image command

```
sudo docker image build -t [image_name] .
```

step 2 :- For save the image

```
sudo docker save -o [file_name.tar] [image_name]
```

step 3 :- For load image

```
sudo docker load -i [filename.tar]
```

For Run Container

step 1 :- Take image or pull it

```
sudo docker pull [image_name]
```

step 2 :- Run below command

```
sudo docker run -d --restart unless-stopped --name microservice_nlp_prediction  
--mount src=/home/ubuntu/models/,target=/models/,type=bind \  
microservice_nlp_prediction:1.0
```

--name microservice_nlp_prediction_container container name

-d detach run in background

-p port 3039 docker port and 8000 project/service port

--mount src = your local models path location

target = inside docker path location note :- don't chane target location

microservice_nlp_prediction:1.0 image name which you pull it in step 1

All option :-

```
sudo docker run -it/-d -p [Docker_Host_Port]:[Container_Port] \  
-v [Docker_Host_Volumn_Dir]:[Container_Volumn_Dir] -e [ENV_name]=[value] \  
--name [container_name] [image_name]:[tag]
```

-it i → interactive, t → terminal

-d detach run in background

-p [Docker_Host_Port]:[Container_Port] → 1234:8000

-e flask environment_variable and value

-v docker volumn directory host and container

--name conatiner name

image name and it's tag

Note: -v and --mount are same not different

method 1: --mount src=/home/host/directory/,target=/home/container/directory/

method 2: -v /home/host/directory:/home/container/directory/

method 3: -v myvolume:/home/container/directory/

myvolume create by docker volume command and it's store in in host /var/lib/v

Docker service Start, Stop, Restart

Start, stop and restart the docker service

```
sudo systemctl start docker.service
```

```
sudo systemctl stop docker.service
```

```
sudo systemctl restart docker.service
```

Docker Basic commands info, inspect

All the information about docker

```
docker info
```

Information about Container

```
docker inspect [container_name/container_id]
```

to check size of container, images and volumes

```
docker system df
```

List Out Images and Containers

check container is up or not

```
sudo docker ps      # it will show only up container
```

```
sudo docker ps -a   # it will show up and down both container
```

List all images

```
sudo docker images
```

Docker Container Start, Stop, Restart

start, stop and restart the container

```
sudo docker start [container_name/container_id]
```

```
sudo docker stop [container_name/container_id]
```

```
sudo docker restart [container_name/container_id]
```

Go inside running Container

attach running container

```
sudo docker attach [container_name/container_id] # it will also show logs
```

```
ctrl + p + q # without stop container exit from that container
```

Inside running container shell/bash

```
sudo docker exec -it [container_name/container_id] bash
```

Remove Containers and Images

remove the container

```
sudo docker rm [container_name/container_id]
```

remove all containers

```
docker rm $(docker ps -aq)
```

remove the image

```
sudo docker rmi [image_name]
```

remove all images

```
docker rmi -f $(docker images -aq)
```

remove overlay Alert

```
sudo docker system prune -a -f # it will remove all the stop containers and images
```

Container Logs

logs of docker container

```
sudo docker logs [container_name/container_id]
```

```
sudo docker container logs -f --tail 10 [container_name/container_id]
```

Create New Image by Commit

Create new Image from existing container

```
docker commit [container_name/container_id] [new_image_name]:[tag]
```

Rename Container, Image

Rename image name

```
sudo docker tag [image_name] [user_name]/[new_image_name]:[tag]
```

Rename container name

```
docker rename [old_name] [new_name]
```

Container Environment Variable option -e

container run with environment variable

```
docker run -e APP_COLOR=blue [image-name] # docker inspect [container_name]
```

Docker Volumes

Create a Volume

```
docker volume create <volume_name>
```

List Volumes

```
docker volume ls
```

Inspect a Volume

```
docker volume inspect <volume_name>
```

Remove a Volume

```
docker volume rm <volume_name>
```

Remove All Unused Volumes

```
docker volume prune
```

Use a Volume in a Container

```
docker run -d -v <volume_name>:<container_path> <image_name>
```

Bind Mounting a Host Directory

```
docker run -d -v /host/path:/container/path <image_name>
```

Bind Mounting a Host Directory

```
docker run -d --mount src=/host/path/,target=/container/path/,type=bind <image>
```

Docker Networks

- Bridge (default): For outside world port mapping.
- None
- Host: For outside world no need port mapping because Docker host and container same port.

```
docker run ubuntu --network=none
```

```
docker run ubuntu --network=host
```

List Docker Networks

```
docker network ls
```

Inspect a Network

```
docker network inspect <network_name>
```

Create a Network

```
docker network create <network_name>
```

Create a Network with Specific Driver

```
docker network create --driver <driver_name> <network_name>
```

Connect a Container to a Network

```
docker network connect <network_name> <container_name_or_id>
```

Remove a Network

```
docker network rm <network_name>
```

Remove All Unused Networks

```
docker network prune
```

Run a Container in a Specific Network

```
docker run --network <network_name> <image_name>
```

Create a Network with Subnet and Gateway

```
docker network create \  
  --driver bridge \  
  --subnet=<subnet> \  
  --gateway=<gateway> \  
  <network_name>
```

DockerFile

app.py

```
import os  
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/')  
def home():  
    return "Hello, World from Dockerized Flask App!"  
  
if __name__ == '__main__':  
    port = int(os.getenv("APP_PORT", 5000)) # Use APP_PORT from env, default 5000  
    app.run(host='0.0.0.0', port=port)
```

requirements.txt

flask

DockerFile

```
# Step 1: Specify the base image
FROM python:3.9-slim
```

```
RUN apt-get update
```

```
# Set environment variables
ENV FLASK_ENV=production \
    APP_PORT=5000
```

```
# Step 2: Set the working directory
WORKDIR /app
```

```
# Step 3: Copy application code into the container
COPY app.py /app/
COPY requirements.txt /app/
```

```
# Step 4: Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

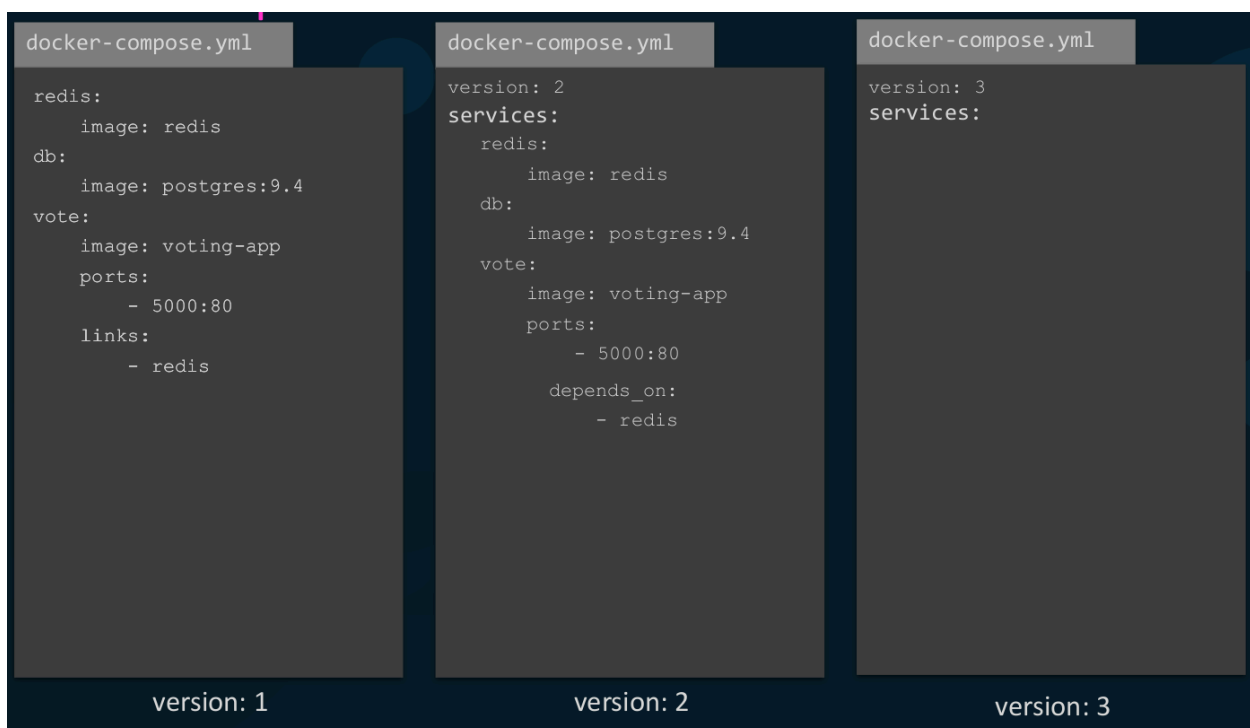
```
# Step 5: Expose the application port
EXPOSE 5000
```

```
# Step 6: Specify the command to run the application
CMD ["python", "app.py"]
# OR
# ENTRYPOINT ["python", "app.py"]
# OR
# ENTRYPOINT ["python"] # Specify the command to run the application
# CMD ["app.py"] # Use CMD to define default arguments for ENTRYPOINT
```

```
docker build -t flask-app .  
docker run -p 5000:5000 -e APP_PORT=5000 flask-app  
sudo docker run -d -p 5000:5000 -e APP_PORT=5000 --mount src=/mnt/newdr
```

Docker Compose

There are three versions of docker-compose.yml file.



- Version 1: if connects container with each other we have to links it.
- Version 2: there is no need to links container because it create it's own network and depends command will launch first depended container like vote depends on redis so first launch redis.

docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  web:
```

```
    image: python:3.9
```

```
    container_name: flask_app
```

```
    working_dir: /app
```

```
    volumes:
```

```
      - ./app:/app
```

```
    ports:
```

```
      - "5000:5000"
```

```
    command: >
```

```
      sh -c "pip install flask && python app.py"
```

```
  db:
```

```
    image: postgres:13
```

```
    container_name: postgres_db
```

```
    environment:
```

```
      POSTGRES_USER: user
```

```
      POSTGRES_PASSWORD: password
```

```
      POSTGRES_DB: example_db
```

```
    ports:
```

```
      - "5432:5432"
```

```
docker-compose up
```

```
docker-compose down
```

```
docker-compose logs
```

```
docker-compose exec
```